AI-Based Natural Language to SQL Query System

Basic idea: Normally, a person needs to know SQL to retrieve information from a database. For example: "Show all CSE students who scored above 80 marks."

The system uses AI to convert this sentence into SQL automatically.

Example: User enters: Find students whose attendance is below 75%.

AI generates:

SELECT StudentName, Attendance FROM Student WHERE Attendance < 75;

The DBMS executes the query and displays the result.

DBMS part

- Student database
- Course database
- Marks database
- Attendance database
- SQL queries
- Joins
- Views
- Constraints
- Database schema

AI part

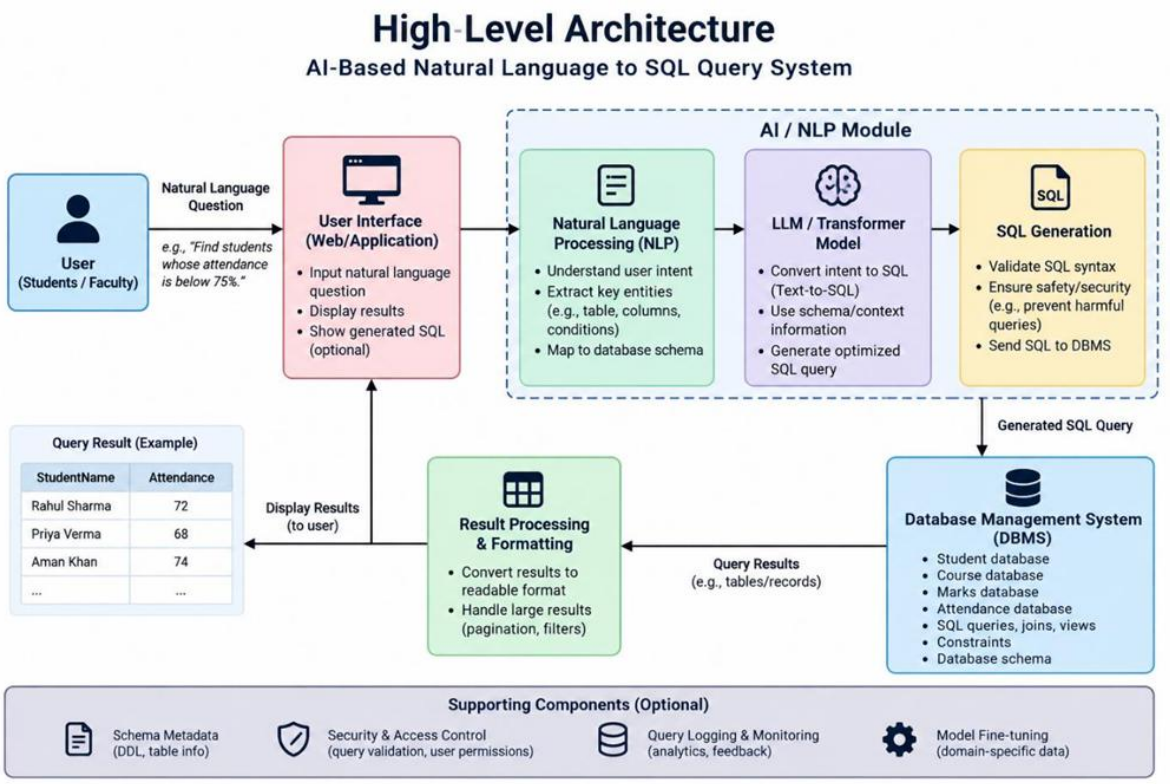
- Natural Language Processing
- Large Language Model / NLP model
- Text-to-SQL generation

Workflow: User Question → AI → SQL Query → DBMS → Result

Algorithms/technology

- NLP
- Transformer/LLM
- Text-to-SQL

Output: A simple interface where students/faculty can ask questions in English instead of writing SQL.



Latency- and Accuracy-Aware Schema Linking for LLM-Based Text-to-SQL Generation

Instead of giving LLM our **entire database schema**, we develop an algorithm that determines:

"For this question, which parts of the database schema actually matter?"

Then LLM only receives those relevant tables/columns.

Advantages of our improved concept:

- Reduced Token Consumption by the LLM
- More context space available to include
- Reduced Latency in the LLM output
- Improved overall efficiency of the system

Several measurable things to optimize:

- **Accuracy**
- **Latency**
- **Prompt/token size**
- **API cost**
- **Schema-linking precision/recall**
- **SQL execution accuracy**

Example User input: “Find me the names of employees whose salary is greater than 80,000.”

Current existing concept: send the entire Employee table schema to the LLM.

Our algorithm: only send the Employee Name, Salary to the LLM to generate the query.

Problem Statement:

Large Language Models can generate SQL queries from natural-language questions, but providing the complete database schema increases input complexity, inference latency, and the possibility of incorrect schema selection. This project investigates whether an intelligent schema-linking and reduction mechanism can improve Text-to-SQL generation accuracy while reducing computational and inference overhead.

if "attendance" in question: select attendance table. Too simple for real world scenario.

Hybrid schema-ranking algorithm:

For every table/column, calculate a relevance score:

$$Score(c,q)=LexicalSimilarity+SemanticSimilarity+TypeMatch+RelationshipScore$$

Research Papers

1. Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task

   https://aclanthology.org/D18-1425/

2. Re-examining the Role of Schema Linking in Text-to-SQL

   https://aclanthology.org/2020.emnlp-main.564/

3. RAT-SQL: Relation-Aware Schema Encoding and Linking for Text-to-SQL Parsers

   https://aclanthology.org/2020.acl-main.677/

4. RESDSQL: Decoupling Schema Linking and Skeleton Parsing for Text-to-SQL

   https://ojs.aaai.org/index.php/AAAI/article/view/26535

5. Can LLM Already Serve as A Database Interface? A Big Bench for Large-Scale Database Grounded Text-to-SQLs
   https://arxiv.org/abs/2305.03111

Quality of Service Parameters targeted to optimize:

Metric	Baseline	Proposed
Schema size	Full	Reduced
Prompt tokens	High	Lower
LLM latency	Measure	Measure
API response time	Measure	Measure
SQL accuracy	Measure	Measure
Execution accuracy	Measure	Measure
Schema-link precision	N/A	Measure
Schema-link recall	N/A	Measure
API cost	Higher	Lower
Local processing time	Very low	Slightly higher